

CareerHub — Autofill Engine Spec

Field detection, matching rules, confidence scoring, error handling & data security

This is the technical rulebook for the auto-fill engine — the single most important and highest-risk feature in CareerHub. It defines exactly how fields are detected, how confidence is scored, what the system is allowed and never allowed to do, and how user data is protected end to end.

1. How the Pipeline Works, End to End

- 1. SCAN -> read all form fields (input, select, textarea, contenteditable) on the page
- 2. EXTRACT -> pull label, name, id, placeholder, aria-label, nearby heading text per field
- 3. MATCH -> compare each field's signals against the user's Profile schema
- 4. SCORE -> assign a confidence score (0-100) per field per candidate Profile key
- 5. DECIDE -> above threshold = auto-fill | below threshold = flag for manual mapping
- 6. FILL -> set value + dispatch real input/change events, staggered per field
- 7. VERIFY -> re-read each field's actual rendered value, compare to intended value
- 8. REPORT -> show "X of Y filled, Z need review" -- never auto-submit

2. Field Detection & Keyword Identification

Signals extracted per field, in priority order:

Signal	Trust level	Example / Notes
1. autocomplete attribute	Highest trust	autocomplete="email", "tel", "given-name" -- these are the field's own declared purpose.
2. name / id attribute	High trust	name="firstName", id="dob_day" -- matched against a normalized alias dictionary.
3. text (for= or wrapping)	High trust	The visible label a human reads, fuzzy-matched against synonyms.
4. placeholder text	Medium trust	Used only if label/name signals are missing or ambiguous.
5. aria-label / aria-labelledby	Medium trust	Common on modern React/Angular forms that skip visible tags.

Signal	Trust level	Example / Notes
6. Nearby section heading	Context modifier	e.g. a heading "Education" above a field raises the score of degree/college/year candidates, lowers score of unrelated ones.
7. Input type / pattern attribute	Constraint filter	type="email", type="tel", pattern="[0-9]{10}" narrow down which Profile keys are even valid candidates.

Keyword normalization (synonym dictionary):

Every Profile field maps to a list of known synonyms/variants seen across portals. Matching is case-insensitive, ignores punctuation/whitespace, and uses fuzzy string distance (e.g. Levenshtein ≤ 2) as a fallback after exact alias lookup fails.

Profile key	Known synonyms/variants matched
phone	Mobile No., Contact Number, Mobile Number, Ph., Cell
dob	Date of Birth, DOB, Birth Date, D.O.B.
fatherName	Father's Name, Father Name, Guardian Name
qualification	Highest Qualification, Educational Qualification, Degree
pincode	PIN Code, Postal Code, ZIP, Pin

3. Confidence Scoring Model

Each field gets a score out of 100 by summing weighted signal matches. This determines whether the engine fills automatically, fills but flags for review, or leaves the field untouched.

Signal combination	Score impact	Rationale
Site-specific pre-built map match	+100 (automatic max)	Known portal, exact selector -- effectively certain.
autocomplete attribute exact match	+45	Field declares its own purpose.
name/id exact alias match	+35	Strong structural signal.
Label text exact/fuzzy match	+30	Fuzzy match below 90% similarity caps contribution at +15.
Placeholder match only	+15	Weak signal alone; combine with others.
Section heading context match	+10	Modifier, not standalone signal.
Input type/pattern conflict	-40	e.g. matched "email" but input type="tel" -- likely wrong, penalize heavily.
No label, no name, no autocomplete	0 (cannot exceed 20 total)	Effectively unidentifiable -- always flagged, never auto-filled.

Decision thresholds:

Score range	Action	Behavior
85-100	Auto-fill silently	Fill immediately, teal flash confirmation.
60-84	Auto-fill + flag for review	Fill the field but mark it amber -- "please double-check".
Below 60	Do not fill	Leave empty, highlight the empty field, prompt user to map manually.

Rule: the engine never fills a field it isn't reasonably confident about, and never treats "filled" as equivalent to "correct." Confidence drives visibility of the review prompt, not just whether to act.

4. Filling Mechanics (Avoiding Silent Failures)

- **Dispatch real events:** after setting a field's value, dispatch native *input* and *change* events (and *keydown/keyup* where a framework listens for them) -- many React/Angular/Vue forms ignore a raw value assignment and will silently revert it otherwise.
- **Match dropdowns by visible text, not value:** select options often have inconsistent internal values across portals ("MH" vs "Maharashtra" vs "27") -- match against the visible option label.

- **Normalize dates to the field's expected format:** detect the portal's format from placeholder/pattern (DD/MM/YYYY vs MM/DD/YYYY vs three separate day/month/year selects) before writing the value.
- **Respect field constraints:** maxlength, pattern, and input masks are read first so the value is pre-formatted to fit (e.g. stripping spaces from a phone number if the field expects digits only).
- **Stagger the fill:** fields populate one after another (~40ms apart) rather than all at once -- both a UX confirmation cue and reduces the chance of triggering anti-bot rate-based detection.

5. Verification & Error Handling

After filling, a verification pass runs automatically:

- Re-read each field's actual DOM value after the fill completes.
- Compare the rendered value against the intended value (accounting for site-side formatting/truncation).
- If they don't match, mark that field amber ("couldn't confirm this filled correctly") rather than reporting false success.
- Summarize results in the on-page chip: "12 of 14 filled - 2 need review" -- always visible, never silent.

If something goes wrong mid-fill:

- **One-click Undo** reverts every field the extension touched in that session back to empty/original.
- **Partial-fill safety:** if the page navigates or reloads mid-fill (common on multi-step forms), the engine stops immediately rather than continuing to fill a now-different page.
- **Duplicate-run guard:** clicking "Use" twice does not double-fill or corrupt already-filled fields; it re-scans current state first.
- **Broken-portal reporting:** a "Report an issue" link on the review chip sends the field-map ID (not personal data) so the central field-map can be patched without an extension update.

6. Absolute Prohibitions (Hard Rules)

These are non-negotiable constraints on the autofill engine, regardless of convenience gains:

- ✗ NEVER auto-submit any form. The user must always click the portal's own submit button.
- ✗ NEVER fill a field scored below the minimum confidence threshold -- leave it empty and flagged instead of guessing.
- ✗ NEVER auto-fill government-ID-style fields (Aadhaar, PAN, passport number) by default -- require explicit per-use confirmation even if stored.
- ✗ NEVER fill payment fields (card numbers, CVV, UPI PIN) under any circumstance -- out of scope entirely, not just low-confidence.
- ✗ NEVER store or transmit field contents to any third-party analytics, ad, or tracking service.
- ✗ NEVER continue a fill operation after detecting page navigation or a DOM structure change mid-run.
- ✗ NEVER present a filled field as verified-correct without the post-fill verification pass actually confirming it.
- ✗ NEVER bypass a portal's own CAPTCHA, bot-detection, or rate limiting -- the extension fills fields a human could type, nothing more.

7. Firm Rules (Always-On Behaviors)

- ✓ ALWAYS show a pre-fill summary (portal name, profile matched, field count) before any field is touched.
- ✓ ALWAYS run the post-fill verification pass and report mismatches honestly.
- ✓ ALWAYS provide one-click Undo for the full fill session.
- ✓ ALWAYS let the user correct a wrong field mapping, and remember the correction for next time on that portal.
- ✓ ALWAYS encrypt Profile data at rest and in transit, with no exceptions for "low-sensitivity" fields.
- ✓ ALWAYS give the user a way to mark specific fields (or whole categories, like ID numbers) as 'never autofill.'

8. Site-Specific Field-Map Schema (Example)

For priority portals, a JSON config (stored and updated server-side, not inside the extension package) maps exact selectors to Profile keys, skipping the scoring step entirely for maximum reliability.

```
{
  "domain": "naukri.com",
  "formVersion": "2026-06",
  "fields": [
    {
      "selector": "#firstName",
      "profileKey": "firstName",
      "matchType": "exact",
      "transform": null
    },
    {
      "selector": "input[name='mobile']",
      "profileKey": "phone",
      "matchType": "exact",
      "transform": "digitsOnly"
    },
    {
      "selector": "select#stateDropdown",
      "profileKey": "state",
      "matchType": "selectByVisibleText",
      "transform": null
    },
    {
      "selector": "input[name='dob']",
      "profileKey": "dob",
      "matchType": "exact",
      "transform": "formatDate:DD/MM/YYYY"
    }
  ],
  "excludeSelectors": [
    "input[name='panNumber']",
    "input[type='password']"
  ]
}
```

The **excludeSelectors** block is deliberate -- it hard-blocks the engine from ever touching listed fields on that portal (IDs, passwords), independent of scoring logic.

9. Data Storage & Encryption Architecture

Layer	Control
At rest	AES-256 encryption on the database; field-level encryption for high-sensitivity items (ID numbers, if ever stored) so a full DB breach doesn't expose them in plaintext even with DB access.
In transit	TLS 1.2+ for all traffic between extension, website, and backend. No field data ever sent over unencrypted channels.

Layer	Control
Extension auth	Short-lived, refreshable session tokens -- never a permanent API key bundled in the extension package (which could be extracted from the extension's own code).
Local caching	Profile data cached in the extension's local storage in encrypted form (not plaintext localStorage), decrypted only in memory at fill-time.
Key management	Encryption keys held in a managed secrets service (e.g. cloud KMS), never hardcoded in source or committed to the repo.
Sensitive-field defaults	Aadhaar/PAN/passport-style fields default to NOT stored; if the user opts in, they're stored with field-level encryption and excluded from autofill by default.
Third parties	Zero analytics/ad SDKs with access to form contents or Profile data; any usage analytics collected must be aggregate/anonymized only.
User control	Full data export (JSON download of their own Profile) and a one-click permanent delete that purges Profile data, cached copies, and backups within a stated retention window.
Breach response	A documented incident-response plan (rotate keys, notify affected users) prepared before launch, not improvised after an incident.

Trust is the entire product. CareerHub's value proposition only works if users believe their resume and personal data are safe. A single data-handling mistake (leak, silent third-party sharing, auto-filling an ID field without consent) can end the product's credibility outright -- security decisions should be treated as product-critical, not a backend afterthought.